Exercise Sheet 1: Convolution, Features, and Visualization

Due on 01.05.2026, 10:00

Thomas Ressler-Antal (t.ressler@lmu.de)

Important Notes:

1. **Email**: Frequently check your email address registered for Moodle. All notifications regarding the course will be sent via Moodle.

2. **Due date**: The exercise sheets will usually be uploaded around 1 week in advance of the due date, which is indicated at the top of the exercise sheet.

3. **Submission**: Your submission should consist of one single ZIP file which includes a PDF file and the corresponding codes. Both the PDF file and ZIP file should contain your surname and your matriculation number (*Surname-MatriculationNumber.zip*) for grading purposes. You may use Jupyter [1] for exporting your python notebooks as PDF, but you still have to hand in your *.ipynb* or *.py* files for us to test your code. For this exercise, please export the *.ipynb* as a PDF file and include that in the ZIP file. **Submissions that fail to follow the naming convention will not be graded.**

This exercise may look intimidating by its length, however, this is because it contains general information about submissions and also many explanations and hints that will help you through the exercise.

General Information:

All programming exercises must be completed in Python. The proposed solution for the exercises will be compiled in Python 3 (3.8 or above). You may use standard Python libraries or Anaconda (open source distribution for Python) to complete your exercises. We will be using PyTorch[2] as our main deep learning framework.

If you have any problems or questions about the exercise, you are welcome to use the student forum on the lecture Moodle page, as most of the time other students might have similar question. For technical issues about the course (for example, in case you cannot upload the solution to Moodle) you can write an email to the person responsible for the exercise (indicated at the top of the exercise sheet).

[1] https://jupyter.org/
[2] https://pytorch.org/

Task 1: Basic Image Manipulation **(4P)**

In this task, you will become familiar with common image processing operations using Python. You may only use the NumPy library, unless specified otherwise.

1. Load the `Wombat.jpg` image using a library of your choice (e.g. Pillow or OpenCV) and convert it to a NumPy array.
2. Print the shape of the image and explain what each dimension represents. Visualize the image and include the shape in the title.
3. Extract a crop of random size centered at the image center and plot it.
4. Randomly permute the color channels (R, G, B) of the crop and plot it.
5. Convert the original image to grayscale and plot it.

   Hint: Grayscaling an image is performed by a weighted average of its color channels. A common choice of weights is:

   $$R : 0.299, \quad G : 0.587, \quad B : 0.114$$

6. Now insert the previously extracted color-permuted crop back into the grayscaled image and plot the result.

   Hint: The grayscale image only has one channel, so you may need to adapt the crop accordingly.

7. Resize the grayscale image with the inserted patch by a factor of 1/4 in width and 1/2 in height.

Always provide clear and descriptive titles for your plots.

Task 2: Convolution and Filtering **(8P)**

Convolution is the central operation behind many classical image processing methods as well as modern convolutional neural networks (CNNs). A convolution can be understood as applying a small matrix of weights (called a *kernel* or *filter*) to an input (e.g., an image).

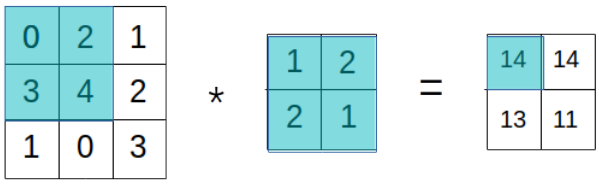

Figure 1: **The convolution operation.** The kernel is slid across the image and applied to local patches. At each position, the output is computed as the dot product between the kernel and the corresponding image patch. Notice how the highlighted value "14" results from $14 = 0 \cdot 1 + 2 \cdot 2 + 3 \cdot 2 + 4 \cdot 1$.

Importantly, the exact same operation is applied at every location in the image. That is, the kernel (i.e. the set of weights) remains fixed while it is shifted across the image. This property is often referred to as **weight sharing**.
This has an important practical consequence: a single kernel defines **one specific operation** (e.g. smoothing, edge detection, or channel mixing) that is applied uniformly across the entire image. Hence, convolution is naturally **translation-invariant**: the same local pattern can be detected or processed regardless of where it appears. Formally, the discrete 2D convolution for a single-channel image is defined as:

$$h(x, y) = \sum_{i=-k}^{k} \sum_{j=-k}^{k} I(x + i, y + j) \cdot K(i, j) \tag{1}$$

In this task, you will first implement convolution for grayscale images and use it for Gaussian smoothing. You will then extend your implementation to multi-channel images and use it to perform channel mixing via a 1×1 convolution.

1. **Single-channel convolution** (2P)

   Implement a convolution function for grayscale images as defined in Eq. 1 and as illustrated in Fig. 1. The function should take a grayscale image of shape (H, W) and a kernel of shape (n, n) as input and should output a filtered image.

   Note: All inputs and outputs should be NumPy arrays. Do not use built-in convolution functions from any library.

2. **Gaussian smoothing** (2P)

   Implement a function that generates a Gaussian kernel of shape (n, n):

$$K(x, y) = \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) \tag{2}$$

   where (x, y) are coordinates relative to the center of the kernel.

   The function should take the kernel size n and the standard deviation σ as inputs, and return the corresponding Gaussian kernel. Normalize the kernel such that all values sum to 1. Plot the resulting kernel as a grayscale image of size 200×200.

   Now apply the kernel to the grayscale version of `Wombat.jpg` using your convolution implementation from the previous subtask. Experiment with different values of the kernel size n and σ, and briefly comment on how these parameters affect the output image.

3. **Extension to multi-channel images** (2P)

   So far, we considered single-channel grayscale images. We now move to multi-channel inputs such as RGB images.

   Extend your convolution implementation to support inputs of shape (H, W, C) and kernels of shape (n, n, C), where the kernel spans all input channels of the image. The output should be a single-channel image obtained by summing over both the spatial dimensions and the channel dimension.

4. **Channel mixing** (1P)

   A special case of convolution is the 1×1 kernel. Such a kernel does not aggregate information across space, but instead mixes the values across channels.

   Construct a 1×1 kernel that converts an RGB image into grayscale and apply it using your multi-channel convolution implementation from the previous subtask. What is the role of the kernel weights in this case?

   Hint: Insights from Task 1 might be useful here.

5. **Discussion** (1P)

   In practice, modern neural networks learn convolutional kernels directly from data instead of defining them manually. Briefly discuss what the limitations of manually designed filters are and what the advantages of learning filters in a data-driven way might be.

Provide visualizations for all relevant results and include descriptive titles.

Task 3: Network Visualization and Interpretation (8P)

In the previous task, you manually designed convolution kernels and studied their effect on an image. In modern convolutional neural networks, however, such filters are learned automatically from data. A natural next question is therefore: *what has the network learned, and which parts of an input image are most relevant for its prediction?*
In this task, you will use a pretrained AlexNet model provided by PyTorch and explore two basic visualization techniques: (i) inspecting the learned filters of the first convolutional layer, and (ii) computing saliency maps that highlight input pixels which most strongly influence a class score.
If you are unfamiliar with loading pretrained models in PyTorch, you may find the official Torchvision documentation helpful: https://docs.pytorch.org/vision/main/models.html. It explains how to load pretrained models and how to apply them to an input image to obtain predictions.

1. **Loading AlexNet** (1P)

   Load a pretrained AlexNet model from `torchvision.models` and print its architecture. Briefly identify the first convolutional layer and state its weight tensor shape. What do the different dimensions of this tensor represent?

   Hint: Use the weights `torchvision.models.AlexNet_Weights.DEFAULT` when loading the AlexNet model.

2. **Visualization of first-layer filters** (2P)

   Extract the learned filters of the first convolutional layer and visualize all of them as images in a grid.

   Since the raw weight values are not directly in a displayable image range, normalize each filter individually before plotting. Provide a plot of all first-layer filters with a descriptive title.

   Hint: The printed model architecture from the previous subtask can help you locate the first convolutional layer.

3. **Discussion of learned filters** (1P)

   Inspect the visualized filters and briefly discuss what you observe. In a few sentences, comment on typical structures that appear in first-layer filters (for example color contrasts, oriented edges, or intensity transitions) and explain why such patterns are useful as early visual features.

4. **Forward pass and prediction** (1P)

Load the image `Wombat.jpg`, preprocess it using the transformations associated with the pretrained AlexNet weights, and pass it through the network.

Determine the predicted class of the image. Which class does the network assign to the input?

Hint: The predicted class index can be mapped to a human-readable label using `weights.meta['categories']`, where `weights` refers to the pretrained AlexNet weights object you loaded earlier.

5. **Gradient computation** (1P)

   Compute the gradient of the predicted class score with respect to the input image pixels.

   Hint: Use `model.zero_grad()` to reset gradients before the model call and use `score.backward()` to backpropagate the class score through the network and obtain gradients with respect to the input image.

   Hint: You will need to enable gradients for the input image.

   Hint: After backpropagation, the gradients of a tensor can be accessed via `tensor.grad` (e.g. for the input image).

6. **Saliency map construction and visualization** (1P)

   Construct a saliency map by taking the absolute value of the gradient and collapsing it over the color channels into a single 2D map (e.g. via a maximum operation). Visualize the resulting saliency map next to the input image.

7. **Discussion** (1P)

   Briefly explain what high saliency values mean in this context. What does the saliency map tell you about the network's decision?

Provide visualizations for all relevant results and include descriptive titles.

Important Note: Submit exactly one ZIP file via Moodle before the deadline. The ZIP file should contain your executable code and your report in PDF format. Make sure that it runs on different operating systems and use relative paths. Non-trivial sections of your code should be explained with short comments, and variables should have self-explanatory names. The PDF file should contain your written code, all

figures, explanations and answers to questions. Make sure that plots have informative axis labels, legends, and captions. Missing plots will result in point reduction.